```
        batch = [self.memory[i] for i in ind] #batch is a list of
tuples
        state1_batch = torch.stack([x[0].squeeze(dim=0) for x in
batch],dim=0)
        action_batch = torch.Tensor([x[1] for x in batch]).long()
        reward_batch = torch.Tensor([x[2] for x in batch])
        state2_batch = torch.stack([x[3].squeeze(dim=0) for x in
batch],dim=0)
        return state1_batch, action_batch, reward_batch,
state2_batch
```

- ***1*** N is the maximum size of the memory list.
- ***2*** batch_size is the number of samples to generate from the memory with the get_batch(...) method.
- ***3*** Every 500 iterations of adding a memory, shuffles the memory list to promote a more random sample
- ***4*** If the memory is not full, adds to the list; otherwise replaces a random memory with the new one
- ***5*** Uses Python's built-in shuffle function to shuffle the memory list
- ***6*** Randomly samples a mini-batch from the memory list
- ***7*** Creates an array of random integers representing indices

The experience replay class essentially wraps a list with extra functionality. We want to be able to add tuples to the list, but only up to a maximum number, and we want to be able to sample from the list. When we sample with the `get_batch(...)` method, we create an array of random integers representing indices in the memory list. We index into the memory list with these indices, retrieving a random sample of memories. Since each sample is a tuple, (S_t,a_t,r_t,S_{t+1}), we want to separate out the different components and stack them together into a S_t tensor, a_t tensor, and so on, where the first dimension of the array is the batch size. For example, the S_t tensor we want to return should be of dimension batch_size × 3 (channels) × 42 (height) × 42 (width). PyTorch's `stack(...)` function will concatenate a list of individual tensors into a single tensor. We also make use of the `squeeze(...)` and `unsqueeze(...)` methods to remove and add dimensions of size `1`.

With all of that set up, we have just about everything we need to train a vanilla DQN besides the training loop itself. In the next section we will